

1 Range queries

1.1 Bit2d

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long
4 using namespace std;
5
6 //kactl
7 struct FT {
8     vector<ll> s;
9     FT(int n) : s(n) {}
10    void update(int pos, ll dif) {
11        for (; pos < s.size(); pos |= pos + 1) s[pos]
12            += dif;
13    }
14    ll query(int pos) {
15        ll res = 0;
16        for (; pos > 0; pos &= pos - 1) res += s[pos
17            - 1];
18        return res;
19    }
20 };
21
22 struct FT2 {
23     vector<vector<ll>> ys; vector<FT> ft;
24     FT2(int limx) : ys(limx) {}
25     void fakeUpdate(int x, int y) {
26         for (; x < ys.size(); x |= x + 1) ys[x].
27             push_back(y);
28     }
29     void init() {
30         for (auto& v : ys) sort(v.begin(), v.end()),
31             ft.emplace_back(v.size());
32     }
33     int ind(int x, int y) {
34         return (int)(lower_bound(ys[x].begin(), ys[x
35             ].end(), y) - ys[x].begin());
36     }
37     void update(int x, int y, ll dif) {
38         for (; x < ys.size(); x |= x + 1)
39             ft[x].update(ind(x, y), dif);
40     }
41     ll query(int x, int y) { //pontos < x e < y
42         ll sum = 0;
43         for (; x; x &= x - 1)
44             sum += ft[x-1].query(ind(x-1, y));
45         return sum;
46     }
47 };

```

1.2 Segment Tree Base

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long
4 using namespace std;
5
6 struct Segtree{
7     vector<ll> segtree;
8     int n;
9     const ll MISS = 0;
10    //nao afetar resultado
11    Segtree(vector<ll> &orig){
12        segtree.resize(4* (orig.size()));
13        n = orig.size();
14        build(1, 0, n-1, orig);
15    }
16    Segtree(int n){
17        segtree.resize(4*n);
18        this->n = n;
19    }

```

```

20
21    ll combine(ll left, ll right){
22        return left + right;
23    }
24
25    //build(0, 0, n-1)
26    void build(int no, int l, int r, vector<ll> &orig
27    ){
28        if(l == r){
29            segtree[no] = orig[l];
30            return;
31        }
32        int mid = (l + r)/2;
33        build(2 * no, l, mid, orig);
34        build((2 * no) + 1, mid + 1, r, orig);
35        segtree[no] = combine(segtree[2 * no],
36            segtree[(2 * no) + 1]);
37    }
38
39    //update(1, 1, n)
40    void update(int no, int l, int r, int pos, ll val
41    ){
42        if((pos < l) or (r < pos)){
43            return;
44        }
45        if(l == r){
46            segtree[no] = val;
47            return;
48        }
49        int mid = (l + r)/2;
50        update(2* no, l, mid, pos, val);
51        update((2* no) + 1, mid + 1, r, pos, val);
52        segtree[no] = combine(segtree[2 * no],
53            segtree[(2 * no) + 1]);
54    }
55
56    //query(1, 1, n)
57    ll query(int no, int l, int r, int lq, int rq){
58        if((rq < l) or (r < lq)){
59            return MISS;
60        }
61        if((lq <= l) and (r <= rq)){
62            return segtree[no];
63        }
64        int mid = (l + r)/2;
65        ll ans = combine(query(2 * no, l, mid, lq, rq
66            ), query((2 * no) + 1, mid + 1, r, lq, rq));
67
68        return ans;
69    }
70 };
71
72 //ADICIONAR X EM L E -X EM R+1 EH IGUAL A ADICIONAR X
73 //EM [L, R]
74 //SEGTRREE PARA ACHAR MENORES:
75 // for(int i = 0; i < n; i++){
76 //     range = upper_bound(mansort.begin(), mansort.
77 //         end(), man[i]) - mansort.begin();
78 //     dir[i] = query(1, 1, n, range, n, segtree);
79 //     update(1, 1, n, range, 1, segtree);
80 // }
81
82 //SegEuler -> usar updates ao inves do build (ou
83 // mapear vetor a[euler_in[i]] -> orig[i]
84 // query em euler_in[i], euler_out[i]
85 // update em euler_in[i]

```

1.3 Mo

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long
4 using namespace std;
5
6
7 const int BLOCK_SZ = 450;
8 //0(n^2/b + q*b)
9 struct Query{
10     int l, r, idx;
11     Query(int li, int ri, int i){
12         l = li; r = ri; idx = i;
13     }
14
15     bool operator<(Query q) const{
16         if((l/BLOCK_SZ) != (q.l/BLOCK_SZ)) return l <
17             q.l;
18         if((l/BLOCK_SZ)&1) return r > q.r;
19         return r < q.r;
20     }
21 };
22 vector<ll> orig;
23 vector<ll> cnt;
24 ll sum = 0;
25 //exemplo elementos distintos
26 void inline add(int idx){
27     cnt[orig[idx]]++;
28     if(cnt[orig[idx]] == 1){
29         sum++;
30     }
31 }
32
33 void inline remove(int idx){
34     cnt[orig[idx]]--;
35     if(cnt[orig[idx]] == 0){
36         sum--;
37     }
38 }
39
40 ll ans(){
41     return sum;
42 }
43
44 void MO(vector<Query> &q, vector<ll> &out){
45     int L = 0, R = 0;
46     sort(q.begin(), q.end());
47     add(0);
48     //queries 0 indexadas
49     for(auto [l, r, idx] : q){
50         while(l < L) add(--L);
51
52         while(r > R) add(++R);
53
54         while(l > L) remove(L++);
55
56         while(r < R) remove(R--);
57
58         out[idx] = ans();
59     }
60 }

```

1.4 Prefix Sum 2d

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 int main(){
6     ios_base::sync_with_stdio(false);

```

```

7     cin.tie(NULL);
8
9     int xi, yi, yf, xf, n, q;
10    vector<vector<int>> prefsum(n + 1, vector<int>(n
11    + 1, 0));
12    vector<vector<int>> floresta;
13    for(int i = 1; i <= n; i++){
14        for(int j = 1; j <= n; j++){
15            prefsum[i][j] = floresta[i - 1][j - 1] +
16                prefsum[i - 1][j] + prefsum[i][j - 1] - prefsum[i
17                - 1][j - 1];
18        }
19    }
20    for(int i = 0; i < q; i++){
21        cin >> yi;
22        cin >> xi;
23        cin >> yf;
24        cin >> xf;
25
26        cout << (prefsum[yf][xf] - prefsum[yf][xi -
27        1] - prefsum[yi - 1][xf] + prefsum[yi - 1][xi -
28        1]) << endl;
29    }
30
31    return 0;
32 }

```

2 Grafos

2.1 Centroid Decomposition

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long
4 using namespace std;
5
6 ll dfs(int u, int p, vector<vector<ll>> &adj, vector<
7 ll> &subtree, vector<bool> &rem){
8     subtree[u] = 1;
9     for(auto v : adj[u]){
10         if(v==p) continue;
11         if(rem[v]) continue;
12         subtree[u] += dfs(v, u, adj, subtree, rem);
13     }
14     return subtree[u];
15 }
16
17 ll centroid(int v, int p, vector<vector<ll>> &adj,
18 vector<ll> &subtree, vector<bool> &rem, int n){
19     for(auto u : adj[v]){
20         if(u==p) continue;
21         if(rem[u]) continue;
22         if(2*subtree[u] > n){
23             return centroid(u, v, adj, subtree, rem,
24             n);
25         }
26     }
27     return v;
28 }
29
30 vector<ll> subtree;
31
32 void decomp(int v, int p, vector<vector<ll>> &adj,
33 vector<bool> &rem){
34     dfs(v, -1, adj, subtree, rem);
35     int cen = centroid(v, -1, adj, subtree, rem,
36     subtree[v]);
37
38     rem[cen] = true;
39
40     for(auto u : adj[cen]){

```

```

36         if(!rem[u]){
37             decomp(u, -1, adj, rem);
38         }
39     }
40 }

```

2.2 Achar Ciclos

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5  int dfs(bool &cicblock, int &cicstart, int cidade,
6          int anterior, vector<int> &ciclo, vector<bool> &
7          visitado, vector<vector<int>> &adj){
8      if(visitado[cidade]){
9          if(cicstart == 0){
10             ciclo.push_back(cidade);
11             cicstart = cidade;
12         }
13         return cidade;
14     } else{
15         int fim = 0;
16         visitado[cidade] = true;
17
18         for(auto i : adj[cidade]){
19             if(i != anterior){
20                 fim = dfs(cicblock, cicstart, i,
21                           cidade, ciclo, visitado, adj);
22             }
23         }
24
25         if(cidade == cicstart){
26             //ciclo.push_back(cidade);
27             cicblock = true;
28         }
29
30         if(fim != -1){
31             if(!cicblock){
32                 ciclo.push_back(cidade);
33                 return cidade;
34             }
35         }
36     }
37
38     return -1;
39 }
40
41 int main() {
42     ios_base::sync_with_stdio(false);
43     cin.tie(NULL);
44
45     //def dfs(atual, anterior):
46     // if(visitado[atual]): return
47     // visitado[atual] = true
48     // for nxt in adj[atual]:
49     //     if(nxt != anterior):
50     //         fim = dfs(nxt, atual)
51     //         if(fim != -1): ciclo.adiciona(atual)
52     //         if(fim == atual OU fim ==
53     JA_TERMINOU): retorne JA_TERMINOU
54     //     retorne -1
55
56     return 0;
57 }

```

2.3 Pontes

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define ll long long int

```

```

4  using namespace std;
5
6  int t = 0;
7  vector<ll> pontes;
8
9  void dfs(int v, int p, vector<vector<tuple<ll, int>>>
10           &adj, vector<bool> &vis, vector<int> &in, vector
11           <int> &low){
12      vis[v] = true;
13      t++;
14      in[v] = t;
15      low[v] = t;
16
17      bool skip = false;
18      for(auto [id, u] : adj[v]){
19          if(u==p && !skip){
20             skip = true;
21         } else{
22             if(vis[u]){
23                 low[v] = min(low[v], in[u]);
24             } else{
25                 dfs(u, v, adj, vis, in, low);
26                 low[v] = min(low[v], low[u]);
27                 if(low[u] > in[v]){
28                     pontes.push_back(id);
29                 }
30             }
31         }
32     }
33 }

```

2.4 Dsu

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5  int find(int n, vector<int> &rep){
6      if(n == rep[n]){
7          return rep[n];
8      } else{
9          rep[n] = find(rep[n], rep);
10         return rep[n];
11     }
12 }
13
14 void join(int n, int v, vector<int> &rep, vector<int>
15           &size){
16     n = find(n, rep);
17     v = find(v, rep);
18
19     if(n == v){
20         return;
21     }
22
23     if(size[n] < size[v]){
24         swap(v, n);
25     }
26
27     rep[v] = n;
28     size[n] += size[v];
29 }
30
31 int main() {
32     ios_base::sync_with_stdio(false);
33     cin.tie(NULL);
34
35     int n, m;
36     vector<int> rep(n + 1), size(n+1);
37     for(int i = 1; i <= n; i++){
38         rep[i] = i;
39         size[i]++;
40     }
41 }

```

```

40
41     return 0;
42 }

```

2.5 Hld

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define ll long long
4  using namespace std;
5
6  struct HLD{
7      int n;
8      vector<vector<int>> adj;
9      vector<int> pai, depth, topo, heavy, pos;
10     int curr = 1;
11     Segtree seg;
12     HLD(int n) : seg(n){
13         this->n = n;
14         adj.resize(n+1);
15         pai.resize(n+1);
16         depth.resize(n+1);
17         topo.resize(n+1);
18         heavy.resize(n+1, -1);
19         pos.resize(n+1);
20     }
21
22     void add(int u, int v){
23         adj[u].push_back(v);
24         adj[v].push_back(u);
25     }
26
27     int find_heavy(int v, int p, int d){
28         int sz = 1;
29         int mx = 0;
30         pai[v] = p;
31         depth[v] = d;
32
33         for(auto u : adj[v]){
34             if(u==p) continue;
35
36             int sub = find_heavy(u, v, d+1);
37             sz += sub;
38             if(sub > mx){
39                 mx = sub;
40                 heavy[v] = u;
41             }
42         }
43
44         return sz;
45     }
46
47     void chains(int v, int p, int ini){
48         topo[v] = ini;
49         pos[v] = curr++;
50
51         if(heavy[v] != -1){
52             chains(heavy[v], v, ini);
53         }
54
55         for(auto u : adj[v]){
56             if(u==p) continue;
57             if(u==heavy[v]) continue;
58
59             chains(u, v, u);
60         }
61     }
62
63     void update(int v, ll val){
64         seg.update(1, 1, n, pos[v], val);
65     }
66
67     ll query_path(int a, int b){

```

```

68         ll res = 0;
69
70         while(topo[a] != topo[b]){
71             if(depth[topo[a]] < depth[topo[b]]){
72                 swap(a, b);
73             }
74             res += seg.query(1, 1, n, pos[topo[a]],
75                             pos[a]);
76             a = pai[topo[a]];
77         }
78
79         if(depth[a] > depth[b]) swap(a, b);
80
81         res += seg.query(1, 1, n, pos[a], pos[b]);
82
83         return res;
84     }
85
86     //tbl tupla bool-ll-bool
87     //exemplo de query path que não é comutativa (
88     // importa a direção)
89     // ll query_path(int a, int b){
90     //     tbl ans_left = seg.MISS;
91     //     tbl ans_right = seg.MISS;
92
93     //     while(topo[a] != topo[b]){
94     //         if(depth[topo[a]] > depth[topo[b]]){
95     //             //subir a
96
97             auto [q0, q1, q2] = seg.query(1,
98             1, n, pos[topo[a]], pos[a]);
99             //CONSIDERAR O REVERSO
100            ans_left = seg.combine(ans_left, {
101            q2, q1, q0});
102
103            a = pai[topo[a]];
104        } else{
105            //subir b
106            tbl qq = seg.query(1, 1, n, pos[
107            topo[b]], pos[b]);
108            ans_right = seg.combine(qq,
109            ans_right);
110
111            b = pai[topo[b]];
112        }
113
114        if(depth[a] > depth[b]){
115            //b é o lca
116            auto [q0, q1, q2] = seg.query(1, 1, n,
117            pos[b], pos[a]);
118            ans_left = seg.combine(ans_left, {q2,
119            q1, q0});
120        } else{
121            tbl q = seg.query(1, 1, n, pos[a], pos
122            [b]);
123            ans_right = seg.combine(q, ans_right);
124        }
125
126        ll ans = get<1>(seg.combine(ans_left,
127        ans_right));
128
129        return ans;
130    }
131
132    void init(){
133        find_heavy(1, -1, 0);
134        chains(1, -1, 1);
135    }
136 }

```

2.6 Dinic

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long int
4 using namespace std;
5
6 struct aresta{
7     ll u, v, c;
8     aresta(){
9         u = 0;
10        v = 0;
11        c = 0;
12    }
13    aresta(ll u2, ll v2, ll c2){
14        u = u2;
15        v = v2;
16        c = c2;
17    }
18 };
19
20 struct Dinic{
21     ll n, src, sink;
22     //vetor all permite achar real e inversa
23     //facilmente
24     vector<aresta> all;
25     vector<vector<int>> adj;
26     vector<ll> level;
27     //proxima aresta disponivel
28     vector<int> nxt;
29
30     Dinic(ll n2, ll src2, ll sink2){
31         n = n2;
32         src = src2;
33         sink = sink2;
34         adj = vector<vector<int>>(n+1);
35     }
36
37     void add(ll u, ll v, ll c, bool dir){
38         //no dinic sao necessarios arestas opostas de
39         //capacidade 0
40         all.push_back(aresta(u, v, c));
41         adj[u].push_back(all.size()-1);
42         //se for nao direcionado trocar 0 por c
43         ll invcap = 0;
44         if(dir){
45             invcap = c;
46         }
47         all.push_back(aresta(v, u, invcap));
48         adj[v].push_back(all.size()-1);
49     }
50
51     //bfs para gerar o vetor lvl por meio de arestas
52     //nao saturadas
53     //deve sinalizar se chegou na pia (caso contrario
54     //o algoritmo terminou)
55     bool bfs(){
56         level = vector<ll>(n+1, -1);
57         queue<tuple<int, ll>> fila;
58         level[src] = 0;
59         fila.push(tuple<int, ll>(src, 0));
60
61         while(!fila.empty()){
62             int curr = get<0>(fila.front());
63             ll d = get<1>(fila.front());
64             fila.pop();
65
66             for(auto e : adj[curr]){
67                 if((all[e].c > 0) && (level[all[e].v]
68                     == -1)){
69                     level[all[e].v] = d+1;
70                     fila.push(tuple<int, ll>(all[e].v,
71                         d+1));
72                 }
73             }
74         }
75
76         return (level[sink] != -1);
77     }
78
79     ll dfs(int curr, ll bottleneck){
80         if((curr == sink) or (bottleneck == 0)){
81             return bottleneck;
82         }
83
84         //atualiza prox aresta, lembrar de resetar
85         for(int &currEdge = nxt[curr]; currEdge < (
86             adj[curr].size()); currEdge++){
87             int ar = adj[curr][currEdge];
88             int nv = all[ar].v;
89             //apenas arestas que progridem
90             if((level[nv] == (level[curr] + 1))){
91                 ll check = dfs(nv, min(bottleneck,
92                     all[ar].c));
93                 if(check > 0){
94                     all[ar].c -= check;
95                     //flipar o ultimo bit checa
96                     inverso
97                     all[ar ^ 1].c += check;
98                     return check;
99                 }
100             }
101         }
102
103         //blocking flow
104         return 0;
105     }
106
107     ll maxflow(){
108         ll flow = 0;
109         while(bfs()){
110             nxt = vector<int>(n+1, 0);
111             while(true){
112                 ll check = dfs(src, 1e12);
113                 if(check > 0){
114                     flow += check;
115                 } else{
116                     break;
117                 }
118             }
119             return flow;
120         }
121     }
122 };
123
124 /*
125 mincut = maxflow
126 apos ultima bfs, level[i] == -1 se esta do lado da
127 fonte
128
129 emparelhamento max:
130 criar src e sink, direcionar arestas src->lado 1,
131 lado 2-> sink
132 todas arestas tem peso 1
133 emp = maxflow
134
135 cobertura minima do bipartido = emparelhamento
136 grids sao bipartidos
137
138 ind - max independent set
139 cob(n) = n - ind(n)
140
141 em um grafo com arestas unitarias:
142 maxflow = quantidade de caminhos aresta-disjuntos s-t
143 cap(corte minimo) = maximo de arestas removidos que

```

```

    quebra os caminhos de s-t
134
135 caminhos vertice-disjuntos = maxflow onde vertices
    sao separados em v1 e v2 com
136 aresta unitaria de v1-v2, v1 onde entra e v2 onde sai
137
138 fechamento maximo: reduzir para min cut
139 lado da fonte: o que adquiriu
140 lado do ralo: o que ignorou
141
142 se valor real i >= 0 : fonte -> i (valor)
143 else i -> ralo (-valor)
144
145 dependencias: se precisa de A pra ter B, aresta: B ->
    A (infinito)
146 */

```

2.7 Euler Tour E Lca

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long
4 using namespace std;
5
6 const int LOG = 20;
7 const int maxn = 2e5 + 10;
8
9 struct LCA{
10     int up[maxn][LOG+1];
11     int n;
12     int tempo = -1; //trocar pra 0 pra virar 1-
        indexado
13     vector<ll> in, out;
14     vector<vector<int>> adj;
15     vector<ll> dst;
16     LCA(int n){
17         memset(up, -1, sizeof up);
18         this->n = n;
19         adj.resize(n+1), in.resize(n+1), out.resize(n
+1), dst.resize(n+1);
20     }
21
22     void add(int u, int v){
23         adj[u].push_back(v);
24         adj[v].push_back(u);
25     }
26
27     void tour(int v, int p, ll d){
28         tempo++;
29         in[v] = tempo;
30
31         dst[v] = d;
32         up[v][0] = p;
33
34         for(auto u : adj[v]){
35             if(u!=p)tour(u, v, d+1);
36         }
37         out[v] = tempo;
38     }
39
40     void build(){
41         for(int j = 1; j <= LOG; j++){
42             for(int i = 1; i <= n; i++){
43                 if(up[i][j-1] != -1){
44                     up[i][j] = up[up[i][j-1]][j-1];
45                 }
46             }
47         }
48     }
49
50     void init(){
51         tour(1, -1, 0);
52         build();

```

```

53     }
54
55     //a Ãl ancestral de b?
56     inline bool is_ancestor(int a, int b){
57         return ((in[a] <= in[b]) && (out[a] >= out[b
]));
58     }
59
60     int lca(int a, int b){
61         if(is_ancestor(a, b)) return a;
62         if(is_ancestor(b, a)) return b;
63
64         int cursor = a;
65         for(int i = LOG; i >= 0; i--){
66             if(up[cursor][i] == -1) continue;
67             if(!(is_ancestor(up[cursor][i], b))){
68                 cursor = up[cursor][i];
69             }
70         }
71
72         return up[cursor][0];
73     }
74
75     ll dist(int a, int b){
76         int lc = lca(a, b);
77
78         return (dst[a] + dst[b] - (2 * dst[lc]));
79     }
80 };

```

2.8 Scc

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long int
4 using namespace std;
5
6 void dfs(int v, vector<vector<int>> &adj, vector<bool>
> &vis, vector<int> &order){
7     vis[v] = true;
8     for(auto u : adj[v]){
9         if(!vis[u]){
10             dfs(u, adj, vis, order);
11         }
12     }
13     order.push_back(v);
14
15     return;
16 }
17
18 void kosaraju(vector<vector<int>> &adj, vector<int> &
c, vector<vector<int>> &comps, vector<vector<int>
>> &dag){
19     int n = adj.size();
20     vector<int> order;
21     vector<bool> vis(n, false);
22
23     for(int i = 1; i < n; i++){
24         if(!vis[i]){
25             dfs(i, adj, vis, order);
26         }
27     }
28
29     vector<vector<int>> adj_transp(n);
30     for(int v = 1; v < n; v++){
31         for(int u : adj[v]){
32             adj_transp[u].push_back(v);
33         }
34     }
35     vis = vector<bool>(n, false);
36     reverse(order.begin(), order.end());
37
38     for(auto v : order){

```

```

39         if(!vis[v]){
40             vector<int> comp;
41             dfs(v, adj_transp, vis, comp);
42             comps.push_back(comp);
43         }
44     }
45
46     vector<bool> mark(comps.size(), false);
47     for(int i = 0; i < comps.size(); i++){
48         for(auto v : comps[i]){
49             c[v] = i;
50         }
51     }
52
53     for(int i = 1; i < n; i++){
54         for(auto v : adj[i]){
55             if(c[i]==c[v] || mark[c[v]]) continue;
56
57             mark[c[v]] = true;
58             dag[c[i]].push_back(c[v]);
59         }
60         for(auto v : adj[i]) mark[c[v]] = false;
61     }
62 }
63
64 int main(){
65     ios_base::sync_with_stdio(false);
66     cin.tie(NULL);
67
68     //2SAT
69     //i = x
70     //i^1 = ~x
71     if(!pos){
72         cout << "IMPOSSIVEL" << endl;
73     } else{
74         vector<bool> val(m+1);
75         for(int i = 1; i <= 2*m + 1; i+=2){
76             val[i/2] = co[i] < co[i^1];
77         }
78         for(int i = 1; i <= m; i++){
79             if(val[i]){
80                 cout << "S ";
81             } else{
82                 cout << "N ";
83             }
84         }
85         cout << endl;
86     }
87 }
88
89 /*
90 void add_or(int u, int v){
91     E[eval(~u)].push_back(eval(v));
92     E[eval(~v)].push_back(eval(u));
93 }
94 void add_nand(int u, int v) {
95     E[eval(u)].push_back(eval(~v));
96     E[eval(v)].push_back(eval(~u));
97 }
98 void set_true (int u){ E[eval(~u)].push_back(eval(u)); }
99 void set_false(int u){ set_true(~u); }
100 void add_imply(int u, int v){ E[eval(u)].push_back(eval(v)); }
101 void add_and (int u, int v){ set_true(u); set_true(v); }
102 void add_nor (int u, int v){ add_and(~u, ~v); }
103 void add_xor (int u, int v){ add_or(u, v); add_nand(u, v); }
104 void add_xnor (int u, int v){ add_xor(u, ~v); }
105 */
106
107

```

```

108     return 0;
109 }

```

2.9 Dijkstra

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int n, m;
10    vector<long long int> distancias(n + 1, LLONG_MAX);
11    vector<vector<tuple<long long int, int>>> adj(n + 1);
12    distancias[1] = 0;
13    priority_queue<tuple<long long int, int>> minheap;
14    minheap.push(tuple(0, 1));
15    long long int d, c;
16    int a, b, o;
17    for(int i = 0; i < m; i++){
18        cin >> a;
19        cin >> b;
20        cin >> c;
21
22        adj[a].push_back(tuple(c, b));
23    }
24    while(minheap.size() > 0){
25        d = -(get<0>(minheap.top()));
26        o = get<1>(minheap.top());
27        minheap.pop();
28
29        if(d <= distancias[o]){
30            for(auto v : adj[o]){
31                if(distancias[get<1>(v)] > distancias[o] + get<0>(v)){
32                    distancias[get<1>(v)] =
33                    distancias[o] + get<0>(v);
34                    minheap.push(tuple(-(distancias[get<1>(v)]), get<1>(v)));
35                }
36            }
37        }
38        // LEMBRE DE GRAPH MODELLING
39        // ADICIONAR ESTADOS COMO EM DP
40
41        return 0;
42    }

```

2.10 Floyd Warshall

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int n, m, a, b;
10    long long int c;
11    vector<vector<long long int>> distancias(n + 1, vector<long long int>(n + 1, LLONG_MAX));
12    for(int i = 0; i < m; i++){
13        cin >> a;
14        cin >> b;
15        cin >> c;

```



```

16 // lembrar arestas duplas
17 distancias[a][b] = min(c, distancias[a][b]);
18 distancias[b][a] = min(c, distancias[b][a]);
19 }
20
21 for(int k = 1; k <= n; k++){
22     for(int u = 1; u <= n; u++){
23         for(int v = 1; v <= n; v++){
24             if((distancias[u][k] != LLONG_MAX)
25 and (distancias[k][v] != LLONG_MAX)){
26                 distancias[u][v] = min(distancias
27 [u][v], (distancias[u][k] + distancias[k][v]));
28             }
29         }
30     }
31 // analisar quantidade, serve pra poucos
32 //Artrices e saber distância entre todas
33
34     return 0;
35 }

```

2.11 Articulações

```

1 #include <bits/stdc++.h>
2 #define ll long long
3 #define endl '\n'
4 using namespace std;
5
6 int t = 0;
7 vector<int> artics;
8 //cuidado com repetidos
9 //inicializar in e low c -1
10
11 void dfs(int v, int p, vector<vector<ll>> &adj,
12 vector<bool> &vis, vector<int> &in, vector<int> &low){
13     vis[v] = true;
14     t++;
15     in[v] = t;
16     low[v] = t;
17
18     int child = 0;
19     for(auto u : adj[v]){
20         if(u==p){
21             continue;
22         }
23         if(vis[u]){
24             low[v] = min(low[v], in[u]);
25         } else{
26             dfs(u, v, adj, vis, in, low);
27             low[v] = min(low[v], low[u]);
28             if(low[u] >= in[v] && p!=-1){
29                 artics.push_back(v);
30             }
31             child++;
32         }
33     }
34     if(p==-1 && child > 1){
35         artics.push_back(v);
36     }
37 }

```

2.12 Bicolorabilidade

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 bool dfs(int v, vector<vector<int>> &adj, vector<bool>
6 > &visitado, vector<bool> &cor){

```

```

6     visitado[v] = true;
7
8     for(auto u : adj[v]){
9         if(!visitado[u]){
10             cor[u] = !cor[v];
11             if(!dfs(u, adj, visitado, cor)){
12                 return false;
13             }
14         } else if(cor[u] == cor[v]){
15             return false;
16         }
17     }
18     return true;
19 }
20
21 void dfs2(int v, long long int dist, vector<bool> &
22 cor, vector<bool> &visitado, vector<vector<tuple<
23 int, long long int>>> &adj){
24     visitado[v] = true;
25     for(auto u : adj[v]){
26         if(!visitado[get<0>(u)]){
27             if((dist + get<1>(u)) % 2 == 0){
28                 cor[get<0>(u)] = cor[1];
29             } else{
30                 cor[get<0>(u)] = !cor[1];
31             }
32             dfs2(get<0>(u), dist + get<1>(u), cor,
33 visitado, adj);
34         }
35     }
36 }
37
38 int main() {
39     ios_base::sync_with_stdio(false);
40     cin.tie(NULL);
41
42     return 0;
43 }

```

2.13 Toposort

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int n, m, a, b;
10     vector<int> grau_entrada(n + 1, 0);
11     vector<vector<int>> requisitos(n + 1);
12     for(int i = 0; i < m; i++){
13         cin >> a;
14         cin >> b;
15         requisitos[a].push_back(b);
16         grau_entrada[b] += 1;
17     }
18
19     queue<int> fila;
20     for(int i = 1; i <= n; i++){
21         if(grau_entrada[i] == 0){
22             fila.push(i);
23         }
24     }
25
26     vector<int> toposort;
27     int u;
28     while(fila.size() > 0){
29         u = fila.front();
30         fila.pop();
31
32         toposort.push_back(u);

```

```

33         for(auto v : requisitos[u]){
34             grau_entrada[v]--;
35             if(grau_entrada[v] == 0){
36                 fila.push(v);
37             }
38         }
39     }
40
41     return 0;
42 }

```

3 Buscas e stl

3.1 Busca Ternaria

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define ll long long
4  using namespace std;
5
6  const ll eps = 2;
7
8  ll calc(ll a);
9
10 int main(){
11     ios_base::sync_with_stdio(false);
12     cin.tie(NULL);
13
14     ll n, a, b;
15     ll r, l;
16     ll m1, m2, f1, f2;
17     while((r-l) > eps){
18         m1 = l + (r-l)/3;
19         m2 = r - (r-l)/3;
20
21         f1 = calc(m1);
22         f2 = calc(m2);
23
24         if(f1 < f2){
25             l = m1;
26         } else{
27             r = m2;
28         }
29     }
30     //SOMENTE PARA INTEIROS
31     ll ans = 0;
32     for(ll i = 1; i <= r; i++){
33         ans = max(ans, (n-i)*a + (i*b + i - ((i*(i+1)
34         ))/2));
35     }
36     //NO CASO DE DOUBLE AJUSTE EPS
37     //OU USE ITERACOES
38     return 0;
39 }

```

3.2 Stack Monotonica

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int n;
10     vector<long long int> tabuas(n);
11     stack<long long int> monotesq;
12     stack<long long int> monotdir;
13     vector<long long int> mindir(n);

```

```

14     vector<long long int> minesq(n);
15
16     for(int i = 0; i < n; i++){
17         while((monotesq.size() > 0) and (tabuas[i] <
18         tabuas[monotesq.top()])){
19             monotesq.pop();
20         }
21
22         if(monotesq.size() > 0){
23             minesq[i] = monotesq.top();
24         } else{
25             minesq[i] = -1;
26         }
27         monotesq.push(i);
28     }
29     //GUARDA MENOR MAIS PROXIMO A ESQUERDA
30
31     for(int i = 1; i <= n; i++){
32         while((monotdir.size() > 0) and (tabuas[n - i
33         ] <= tabuas[monotdir.top()])){
34             monotdir.pop();
35         }
36
37         if(monotdir.size() > 0){
38             mindir[n - i] = monotdir.top();
39         } else{
40             mindir[n - i] = n;
41         }
42         monotdir.push((n - i));
43     }
44     //VERSAO INVERTIDA
45
46     long long int ar;
47     long long int maxarea = 0;
48     for(int i = 0; i < n; i++){
49         ar = (mindir[i] - minesq[i] - 1) * tabuas[i];
50         maxarea = max(ar, maxarea);
51     }
52     cout << maxarea << endl;
53
54     return 0;
55 }

```

3.3 Kadane

```

1  #include <bits/stdc++.h>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int len, elemento;
10     long long maxsum, prevsum;
11     int arr[len];
12     prevsum = arr[0];
13     maxsum = arr[0];
14     //maior soma em subarray
15     for (int j = 1; j < len; j++){
16         if((prevsum + arr[j]) < arr[j]){
17             prevsum = arr[j];
18         } else{
19             prevsum += arr[j];
20         }
21
22         if (prevsum > maxsum){
23             maxsum = prevsum;
24         }
25     }
26     cout << maxsum << endl;
27

```

```
28     return 0;
29 }
```

4 String

4.1 Trie

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 struct Node{
6     int next[26];
7     int subtree = 0;
8 };
9
10 void add(string s, vector<Node> &trie){
11     int curr = 0;
12
13     for(auto c : s){
14         if(trie[curr].next[c - 'a'] == 0){
15             trie[curr].next[c - 'a'] = trie.size();
16             trie.push_back(Node());
17         }
18
19         trie[curr].subtree += 1;
20         curr = trie[curr].next[c - 'a'];
21     }
22     trie[curr].subtree += 1;
23 }
24
25 int query(string s, vector<Node> &trie){
26     int curr = 0;
27
28     for(auto c : s){
29         if(trie[curr].next[c - 'a'] == 0){
30             return 0;
31         }
32         curr = trie[curr].next[c - 'a'];
33     }
34     return trie[curr].subtree;
35 }
36
37 int main() {
38     ios_base::sync_with_stdio(false);
39     cin.tie(NULL);
40
41     vector<Node> trie(1);
42     //trie pode ser modificada com DFS para propagar
43     //mudancas
44     //TRIE DE XOR -> max(busca diferentes) e min(
45     //busca igual)
46     return 0;
47 }
```

4.2 Aho Corasick

```
1 #include <bits/stdc++.h>
2 #define ll long long
3 using namespace std;
4
5 struct AhoCorasick {
6     static const int K = 26; //tamanho do alfabeto
7     static const char offset = 'a';
8
9     struct Node {
10         int next[K];
11         bool isword = false;
12         int sflink = -1; //aponta para no do maior
13         //sufixo que eh prefixo de alguma palavra do
14         //dicionario
15     };
16
17     vector<Node> trie;
18
19     void add(string s){
20         int curr = 0;
21
22         for(auto c : s){
23             if(trie[curr].next[c - offset] == -1){
24                 trie[curr].next[c - offset] = trie.size();
25                 trie.push_back(Node());
26             }
27
28             trie[curr].sflink += 1;
29             curr = trie[curr].next[c - offset];
30         }
31         trie[curr].isword = true;
32     }
33
34     void build(){
35         queue<int> fila;
36
37         for(int i = 0; i < K; i++){
38             if(trie[0].next[i] == -1){
39                 trie[0].next[i] = 0;
40             } else{
41                 trie[trie[0].next[i]].sflink = 0;
42                 trie[trie[0].next[i]].exlink = 0;
43                 fila.push(trie[0].next[i]);
44             }
45         }
46
47         while(!fila.empty()){
48             auto v = fila.front();
49             fila.pop();
50
51             int go = trie[v].sflink;
52             if(trie[go].isword){
53                 trie[v].exlink = go;
54             } else{
55                 trie[v].exlink = trie[go].exlink;
56             }
57
58             //usar build para acumular coisas pra
59             //query
60             //como dp
61             //exemplo: quantidade de ocorrencias
62             trie[v].cnt += trie[trie[v].sflink].cnt;
63
64             for(int i = 0; i < K; i++){
65                 if(trie[v].next[i] == -1){
66                     trie[v].next[i] = trie[trie[v].
67                     sflink].next[i];
68                 } else{
69                     trie[trie[v].next[i]].sflink =
70                     trie[trie[v].sflink].next[i];
71                     fila.push(trie[v].next[i]);
72                 }
73             }
74         }
75     }
76 }
```

```
13     int exlink = -1; //aponta para no do maior
14     //sufixo que eh uma palavra do dicionario
15     vector<int> idxs; //indices da palavra (se
16     //forem unicas trocar pra int)
17     int cnt = 0;
18
19     Node() {
20         fill(begin(next), end(next), -1);
21     }
22
23     vector<Node> trie;
24
25     AhoCorasick(){
26         trie.push_back(Node());
27     }
28
29     void add(string &s, int id){
30         int curr = 0;
31         for(auto c : s){
32             if(trie[curr].next[c - offset] == -1){
33                 trie[curr].next[c - offset] = trie.size();
34                 trie.push_back(Node());
35             }
36             curr = trie[curr].next[c - offset];
37         }
38         trie[curr].isword = true;
39         trie[curr].cnt++;
40         trie[curr].idxs.push_back(id);
41     }
42
43     void build(){
44         queue<int> fila;
45
46         for(int i = 0; i < K; i++){
47             if(trie[0].next[i] == -1){
48                 trie[0].next[i] = 0;
49             } else{
50                 trie[trie[0].next[i]].sflink = 0;
51                 trie[trie[0].next[i]].exlink = 0;
52                 fila.push(trie[0].next[i]);
53             }
54         }
55
56         while(!fila.empty()){
57             auto v = fila.front();
58             fila.pop();
59
60             int go = trie[v].sflink;
61             if(trie[go].isword){
62                 trie[v].exlink = go;
63             } else{
64                 trie[v].exlink = trie[go].exlink;
65             }
66
67             //usar build para acumular coisas pra
68             //query
69             //como dp
70             //exemplo: quantidade de ocorrencias
71             trie[v].cnt += trie[trie[v].sflink].cnt;
72
73             for(int i = 0; i < K; i++){
74                 if(trie[v].next[i] == -1){
75                     trie[v].next[i] = trie[trie[v].
76                     sflink].next[i];
77                 } else{
78                     trie[trie[v].next[i]].sflink =
79                     trie[trie[v].sflink].next[i];
80                     fila.push(trie[v].next[i]);
81                 }
82             }
83         }
84     }
85 }
```

```

80     }
81 }
82
83 //exemplo: quantidade de matches
84 ll query_matches(string &s){
85     ll ans = 0;
86     int curr = 0;
87
88     for(auto c : s){
89         curr = trie[curr].next[c-offset];
90         ans += trie[curr].cnt;
91     }
92
93     return ans;
94 }
95
96 //exemplo: indices dos matches
97 vector<int> query_indexes(string &s){
98     vector<int> ans;
99
100    vector<bool> vis(trie.size());
101    int curr = 0;
102
103    for(auto c : s){
104        curr = trie[curr].next[c-offset];
105
106        int up = curr;
107        while((up > 0) && !vis[up]){
108            vis[up] = true;
109
110            if(trie[up].isword){
111                for(auto id : trie[up].idxs){
112                    ans.push_back(id);
113                }
114            }
115
116            up = trie[up].exlink;
117        }
118    }
119
120    return ans;
121 }
122
123 };

```

4.3 Suffix Array

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long int
4 using namespace std;
5
6 struct SA{
7     int sz;
8     vector<ll> suffarr, endrank, lcp;
9     SA(int s){
10         sz = s;
11         suffarr = vector<ll>(sz+1); //espaço p/
12         sentinela vai ser apagado
13         endrank = vector<ll>(sz);
14         lcp = vector<ll>(sz);
15     }
16     void suffarray(string &s){
17         //ajuste a sentinela se necessario
18         s.push_back(char(0));
19         const ll n = s.size();
20         vector<ll> rank(n), temp(n);
21         vector<pair<ll, ll>> etc(n);
22         for(int i = 0; i < n; i++){
23             etc[i] = {s[i], i};
24         }
25         sort(etc.begin(), etc.end());
26         for(int i = 0; i < n; i++){

```

```

26             suffarr[i] = etc[i].second;
27         }
28         rank[suffarr[0]] = 0;
29         for(int i = 1; i < n; i++){
30             rank[suffarr[i]] = rank[suffarr[i-1]] + (
31                 etc[i].first != etc[i-1].first);
32         }
33
34         for(int k = 1; k < n; k *= 2){
35             int classes = rank[suffarr[n-1]] + 1;
36             vector<ll> cnt(classes, 0);
37             for(int i = 0; i < n; i++) cnt[rank[i
38                 ]++]++;
39
40             vector<ll> pos(classes, 0);
41             for(int i = 1; i < classes; i++) pos[i] =
42                 pos[i-1] + cnt[i-1];
43
44             for(int i = 0; i < n; i++){
45                 int j = suffarr[i] - k;
46                 if(j < 0) j+=n;
47                 temp[pos[rank[j]]++] = j;
48             }
49             suffarr = temp;
50
51             temp[suffarr[0]] = 0;
52             for(int i = 1; i < n; i++){
53                 pair<ll, ll> prev = {rank[suffarr[i
54                     -1]], rank[(suffarr[i-1] + k) % n]};
55                 pair<ll, ll> curr = {rank[suffarr[i
56                     ]], rank[(suffarr[i] + k) % n]};
57
58                 temp[suffarr[i]] = temp[suffarr[i-1]]
59                     + (curr != prev);
60             }
61             rank = temp;
62         }
63         s.pop_back();
64         suffarr.erase(suffarr.begin());
65         return;
66     }
67
68     //lcp[i] = lcp de i-1 e i; lcp[0] deve ser
69     ignorado
70     void lcpv(string &s){
71         s.push_back(char(0));
72         int n = suffarr.size();
73
74         for(int i = 0; i < n; i++){
75             endrank[suffarr[i]] = i;
76         }
77         ll k = 0;
78
79         for(int i = 0; i < n; i++){
80             if(endrank[i] == 0) lcp[0] = 0;
81             else{
82                 ll j = suffarr[endrank[i] - 1];
83                 while(s[i+k] == s[j+k]) k++;
84                 lcp[endrank[i]] = k;
85                 if(k > 0) k--;
86             }
87         }
88         s.pop_back();
89         return;
90     }
91
92     //retorna tamanho do lcp entre sufixos da posicao
93     i e j
94     //so pra mostrar que eh uma query de minimo no
95     lcp
96     //ao inves disso use seg
97     ll get_lcp(int i, int j){
98         int l = endrank[i];

```

```

90     int r = endrank[j];
91     if(l > r) swap(l, r);
92     if(i==j) return (sz - suffarr[l]);
93
94     ll ans = lcp[l+1];
95     for(int k = l+2; k <= r; k++){
96         ans = min(lcp[k], ans);
97     }
98     return ans;
99 }
100 };

```

4.4 Hash Sem Ordem

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(NULL);
10
11      const long long int k = 277;
12      const long long int l = 149;
13      const long long int p = 37;
14
15      const long long int mod = 1000000009;
16      const long long int mod2 = 1000000007;
17      const long long int mod3 = 999999937;
18
19      int n, q;
20      cin >> n;
21      cin >> q;
22      vector<long long int> num(n);
23      vector<long long int> bnum(n);
24      vector<long long int> pot(n+1);
25      vector<long long int> pot2(n+1);
26      vector<long long int> pot3(n+1);
27      pot[0] = 1;
28      pot[1] = 1;
29      pot[2] = 1;
30      for(int po = 1; po <= n; po++){
31          pot[po] = (pot[po - 1] * k) % mod;
32          pot2[p] = (pot2[po - 1] * l) % mod2;
33          pot3[po] = (pot3[po - 1] * p) % mod3;
34      }
35      for(int i = 0; i < n; i++){
36          cin >> num[i];
37      }
38      for(int i = 0; i < n; i++){
39          cin >> bnum[i];
40      }
41      vector<long long int> hashsuml(n+1);
42      vector<long long int> hashsumk(n+1);
43      vector<long long int> hashsump(n+1);
44      hashsumk[0] = 0;
45      hashsuml[0] = 0;
46      hashsump[0] = 0;
47      hashsumk[1] = (num[0] * pot[num[0]]) % mod;
48      hashsuml[1] = (num[0] * pot2[num[0]]) % mod2;
49      hashsump[1] = (num[0] * pot3[num[0]]) % mod3;
50
51      vector<long long int> bhashsuml(n+1);
52      vector<long long int> bhashsumk(n+1);
53      vector<long long int> bhashsump(n+1);
54      bhashsumk[0] = 0;
55      bhashsuml[0] = 0;
56      bhashsump[0] = 0;
57      bhashsumk[1] = (bnum[0] * pot[bnum[0]]) % mod;
58      bhashsuml[1] = (bnum[0] * pot2[bnum[0]]) % mod2;
59      bhashsump[1] = (bnum[0] * pot3[bnum[0]]) % mod3;

```

```

60     for(int i = 1; i < n; i++){
61         hashsumk[i+1] = ((hashsumk[i] + ((num[i] *
62             pot[num[i]]) % mod)) % mod);
63         hashsuml[i+1] = ((hashsuml[i] + ((num[i] *
64             pot2[num[i]]) % mod2)) % mod2);
65         hashsump[i+1] = ((hashsump[i] + ((num[i] *
66             pot3[num[i]]) % mod3)) % mod3);
67     }
68     for(int i = 1; i < n; i++){
69         bhashsumk[i+1] = ((bhashsumk[i] + ((bnum[i] *
70             pot[bnum[i]]) % mod)) % mod);
71         bhashsuml[i+1] = ((bhashsuml[i] + ((bnum[i] *
72             pot2[bnum[i]]) % mod2)) % mod2);
73         bhashsump[i+1] = ((bhashsump[i] + ((bnum[i] *
74             pot3[bnum[i]]) % mod3)) % mod3);
75     }
76     return 0;
77 }

```

4.5 Hash Bom

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define ll long long
4  using namespace std;
5  using Hash = pair<ll, ll>;
6
7  struct Hasher {
8      static const ll P1 = 31;
9      static const ll P2 = 37;
10     static const ll M1 = 1e9 + 33;
11     static const ll M2 = 1e9 + 93;
12
13     ll n;
14     vector<ll> prefix_hash1, prefix_hash2;
15     static vector<ll> powers1, powers2;
16
17     static void ensure_powers(ll max_len) {
18         ll current_size = powers1.size();
19         if (max_len > current_size) {
20             powers1.resize(max_len);
21             powers2.resize(max_len);
22             for (int i = current_size; i < max_len; i
23 ++) {
24                 powers1[i] = (powers1[i - 1] * P1) %
25 M1;
26                 powers2[i] = (powers2[i - 1] * P2) %
27 M2;
28             }
29         }
30     }
31     Hasher(const vector<ll>& s) {
32         n = s.size();
33         if (powers1.empty()) {
34             powers1.push_back(1);
35             powers2.push_back(1);
36         }
37         ensure_powers(n + 1);
38
39         prefix_hash1.resize(n + 1, 0);
40         prefix_hash2.resize(n + 1, 0);
41
42         for (int i = 0; i < n; i++) {
43             ll val = s[i];
44             prefix_hash1[i + 1] = (prefix_hash1[i] *
45 P1 + val) % M1;
46             prefix_hash2[i + 1] = (prefix_hash2[i] *
47 P2 + val) % M2;
48         }
49     }

```

```

47 Hash substr_hash(int i, int j) {
48     int len = j - i + 1;
49
50     ll h1 =
51         (((prefix_hash1[j + 1] - (prefix_hash1[i]
52 * powers1[len]) % M1) %
53         M1) +
54         M1) %
55         M1;
56
57     ll h2 =
58         (((prefix_hash2[j + 1] - (prefix_hash2[i]
59 * powers2[len]) % M2) %
60         M2) +
61         M2) %
62         M2;
63
64     return {h1, h2};
65 }
66
67 Hash full_hash() {
68     if (n == 0) {
69         return {0, 0};
70     }
71     return substr_hash(0, n - 1);
72 }
73
74 vector<ll> Hasher::powers1;
75 vector<ll> Hasher::powers2;

```

5 Dp

5.1 Lcs

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 5*1e3 + 5;
5 int memo[MAXN][MAXN];
6
7 string s, t;
8
9 inline int LCS(int i, int j){
10     if(i == s.size() || j == t.size()) return 0;
11     if(memo[i][j] != -1) return memo[i][j];
12
13     if(s[i] == t[j]) return memo[i][j] = 1 + LCS(i+1,
14 j+1);
15
16     return memo[i][j] = max(LCS(i+1, j), LCS(i, j+1));
17 }
18
19 int LCS_It(){
20     for(int i=s.size()-1; i>=0; i--){
21         for(int j=t.size()-1; j>=0; j--){
22             if(s[i] == t[j])
23                 memo[i][j] = 1 + memo[i+1][j+1];
24             else
25                 memo[i][j] = max(memo[i+1][j], memo[
26 i][j+1]);
27
28             return memo[0][0];
29 }
30
31 string RecoverLCS(int i, int j){
32     if(i == s.size() || j == t.size()) return "";
33
34     if(s[i] == t[j]) return s[i] + RecoverLCS(i+1, j
35 +1);
36
37 }

```

```

34     if(memo[i+1][j] > memo[i][j+1]) return RecoverLCS
35 (i+1, j);
36
37     return RecoverLCS(i, j+1);
38 }
39 //creditos para SamuelH12
40 //*****
41 LCS - Longest Common Subsequence
42
43 Complexity: O(N^2)
44
45 * Recursive:
46 memset(memo, -1, sizeof memo);
47 LCS(0, 0);
48
49 * Iterative:
50 LCS_It();
51
52 * RecoverLCS
53 Complexity: O(N)
54 Recover one of all the possible LCS
55 *****/

```

5.2 Lcis

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int n, pos;
10    cin >> n;
11    vector<int> v(n);
12    for(int i = 0; i < n; i++){
13        cin >> v[i];
14    }
15    vector<int> aux;
16    aux.push_back(v[0]);
17    for(int i = 1; i < n; i++){
18        pos = upper_bound(aux.begin(), aux.end(), v[i
19 ]) - aux.begin();
20        if(pos >= aux.size()){
21            if(v[i] != aux[pos - 1]){
22                aux.push_back(v[i]);
23            }
24        } else{
25            if(v[i] != aux[pos - 1]){
26                aux[pos] = v[i];
27            }
28        }
29    }
30    cout << aux.size() << endl;
31
32    return 0;
33 }

```

5.3 Operacoes-bitwise

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5
6 int main() {
7     ios_base::sync_with_stdio(false);
8     cin.tie(NULL);
9
10    //2^n = (1<n)
11    int n, i, mask;

```

```

12     for(int mask = 0; mask <(1<<n); mask++);
13     //iterar pela mask n
14
15     if(mask&(1<<i)); //se bit i for 1
16     mask = mask|(1<<i); //ligar bit i
17     mask = mask^(1<<i); //flipar bit i
18
19     return 0;
20 }

```

5.4 Digit Dp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5  //dp para numeros sem 2 digitos adjacentes iguais
6  long long int rec(int pos, int lastdig, bool start,
7      bool smaller, vector<int> &num, vector<vector<
8      vector<vector<long long int>>>> &dp){
9      if(pos >= num.size()){
10         return 1;
11     }
12
13     if(dp[pos][lastdig][start][smaller] != -1){
14         return dp[pos][lastdig][start][smaller];
15     }
16
17     long long int ans = 0;
18     if(smaller){
19         for(int i = 0; i <= 9; i++){
20             if(start){
21                 if(i > 0){
22                     ans += rec(pos + 1, i, false,
23                         smaller, num, dp);
24                 } else{
25                     ans += rec(pos + 1, i, true,
26                         smaller, num, dp);
27                 }
28             } else{
29                 if(i != lastdig){
30                     ans += rec(pos + 1, i, start,
31                         smaller, num, dp);
32                 }
33             }
34         }
35     } else{
36         for(int i = 0; i <= num[pos]; i++){
37             if(start){
38                 if(i > 0){
39                     ans += rec(pos + 1, i, false, !(i
40                         == num[pos]), num, dp);
41                 } else{
42                     ans += rec(pos + 1, i, true, !(i
43                         == num[pos]), num, dp);
44                 }
45             } else{
46                 if(i != lastdig){
47                     ans += rec(pos + 1, i, start, !(i
48                         == num[pos]), num, dp);
49                 }
50             }
51         }
52     }
53     dp[pos][lastdig][start][smaller] = ans;
54     return dp[pos][lastdig][start][smaller];
55 }
56
57 int main() {
58     ios_base::sync_with_stdio(false);
59     cin.tie(NULL);
60 }

```

```

54     //digit dp: iterar pelos digitos
55     //lembrar da ideia base: quando chegar na
56     //posiçãõ
57     //nãõ pode mudar o valor!!
58     return 0;
59 }

```

5.5 Sos Dp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define ll long long
4  using namespace std;
5
6  const int bits = 22;
7  ll dp[111 << bits];
8  void sosdp(bool submask){
9      memset(dp, 0, sizeof dp);
10
11      if(submask){
12
13          //dp[mask] guarda soma de dp[] de todas as
14          SUBmasks
15          for(int i = 0; i < bits; i++){
16              for(ll mask = 0; mask < (111 << bits);
17              mask++){
18                  if(mask&(111 << i)) dp[mask] += dp[
19                      mask^(111 << i)];
20              }
21          } else{
22
23              //dp[mask] guarda a soma de dp[] de todas as
24              SUPERmasks
25              for(int i = 0; i < bits; i++){
26                  for(ll mask = (111 << bits) - 1; mask >=
27                  0; mask--){
28                      if(!(mask & (111 << i))) {
29                          dp[mask] += dp[mask ^ (111 << i)
30                      ];
31                  }
32              }
33          }
34      }
35  }
36  }
37  }
38  }
39  }
40  }
41  }
42  }
43  }
44  }
45  }
46  }
47  }
48  }
49  }
50  }
51  }
52  }
53  }
54  }
55  }
56  }
57  }
58  }
59  }
60  }
61  }
62  }
63  }
64  }
65  }
66  }
67  }
68  }
69  }
70  }
71  }
72  }
73  }
74  }
75  }
76  }
77  }
78  }
79  }
80  }
81  }
82  }
83  }
84  }
85  }
86  }
87  }
88  }
89  }
90  }
91  }
92  }
93  }
94  }
95  }
96  }
97  }
98  }
99  }
100 }

```

5.6 Knapsack 2d

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5  long long int knapsack(vector<tuple<long long int,
6  long long int>> &itens, vector<vector<long long
7  int>> &dp, long long int w, int i){
8      if(w == 0){
9          return 0;
10     }
11     if(i >= itens.size()){
12         return 0;
13     }
14     if(dp[w][i] != -1){
15         return dp[w][i];
16     }
17
18     long long int ans = knapsack(itens, dp, w, i + 1)
19     ;
20     if(get<0>(itens[i]) <= w){
21         ans = max(ans, (get<1>(itens[i]) + knapsack(
22             itens, dp, (w - get<0>(itens[i])), i + 1)));
23     }
24 }
25
26 int main() {
27     ios_base::sync_with_stdio(false);
28     cin.tie(NULL);
29 }

```

```

19     }
20     dp[w][i] = ans;
21     return ans;
22 }
23
24 int main() {
25     ios_base::sync_with_stdio(false);
26     cin.tie(NULL);
27
28     return 0;
29 }

```

5.7 Knapsack 1d

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 long long int knap(map<int, int> &custo, vector<long
    long int> &dp, int w){
6     if(w == 0){
7         return 0;
8     }
9
10    if(dp[w] != -1){
11        return dp[w];
12    }
13
14    long long int ans = 0;
15    for(auto i : custo){
16        if((w - i.first) >= 0){
17            ans = max(ans, (knap(custo, dp, (w - i.
18                first))) + i.second);
19        }
20    }
21    dp[w] = ans;
22    return dp[w];
23 }
24
25 int main() {
26     ios_base::sync_with_stdio(false);
27     cin.tie(NULL);
28
29     //mesmos principios da digit, lembre-se do kongey
30     donk
31     //estados, mudan as, dp[i][j] -> dp[i- 1][j], dp
32     [i - 1][j + 1], dp[i - 1][j - 1]
33
34     return 0;
35 }

```

6 Matematica

6.1 Divisores

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int n;
10    vector<int> divs;
11    for(int i = 1; (i * i) <= n; i++){
12        if(n % i == 0){
13            divs.push_back(i);
14            if(i != n/i){
15                divs.push_back(n/i);

```

```

16            }
17        }
18    }
19
20    return 0;
21 }

```

6.2 Fexp E Comuns

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 const long long int mod = 1000000007;
6
7 long long int fexp(long long int a, long long int b){
8     long long int ans = 1;
9     while(b != 0){
10        if(b & 1){
11            ans = (ans * a) % mod;
12        }
13        a = (a * a) % mod;
14        b >>= 1;
15    }
16    return ans;
17 }
18
19 long long int gcd(long long int a, long long int b){
20     if(!b){
21         return a;
22     } else{
23         return gcd(b, a % b);
24     }
25     //ja implementado em __gcd()
26 }
27
28 long long int lcm(long long int a, long long int b){
29     return ((a*b)/gcd(a,b));
30 }
31
32 int main() {
33     ios_base::sync_with_stdio(false);
34     cin.tie(NULL);
35
36     return 0;
37 }

```

6.3 Combinacao

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 const long long int mod = 1000000007;
6
7 long long int comb(long long int n, long long int i){
8     long long int denom = 1;
9     long long int num = 1;
10
11    for(int j = 0; j < i; j++){
12        num *= (n - j);
13        num /= (j + 1);
14    }
15    //COMBINACAO ITERATIVA
16    return num;
17 }
18
19 long long int fexp(long long int a, long long int b);
20
21 int main() {
22     ios_base::sync_with_stdio(false);

```



```

23 cin.tie(NULL);
24
25 long long int n, m;
26 vector<long long int> fatn(2000100);
27 fatn[0] = 1;
28 for(long long int i = 1; i < fatn.size(); i++){
29     fatn[i] = (fatn[i - 1] * i) % mod;
30 }
31 //combinacao = (n!/(i!*(n-i)))
32 //combinacao com repeticao C(n, i) = C(n + i - 1, i);
33 long long int aa = ((fatn[m] * fatn[n - 1])) % mod;
34 long long int bb = fexp(aa, mod - 2);
35 long long int combrep = (fatn[n + m - 1] * bb) % mod;
36
37 long long int comb = ((fatn[n])/(fatn[n-m] * fatn[m]));
38 //para operacoes com modulo eh preciso ao inves de dividir, multiplicar
39 //pelo inverso modular (n ^ mod-2)
40
41 return 0;
42 }

```

6.4 Modulo Struct

```

1 #include <bits/stdc++.h>
2 #define ll long long
3 #define endl '\n'
4 using namespace std;
5
6 const ll mod = 1e9 + 7;
7
8 struct mint{
9     ll val;
10    mint(){
11        this->val = 0;
12    }
13    mint(ll val){
14        this->val = ((val % mod) + mod) % mod;
15    }
16
17    mint operator+(mint v){
18        return (((val + v.val) % mod) + mod) % mod;
19    }
20    mint operator-(mint v){
21        return (((val - v.val) % mod) + mod) % mod;
22    }
23    mint operator*(mint v){
24        return (((val * v.val) % mod) + mod) % mod;
25    }
26    friend ostream& operator<<(ostream& os, const mint& m) {
27        return os << m.val;
28    }
29
30    friend istream& operator>>(istream& is, mint& m)
31    {
32        ll x;
33        is >> x;
34        m = mint(x);
35        return is;
36    }
37 };

```

6.5 Crivo

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;

```

```

4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(NULL);
8
9     int lim;
10    vector<bool> isprime;
11    isprime[0] = false;
12    isprime[1] = false;
13    vector<int> primes;
14    //crivo base, acha primos
15    for(int i = 2; i < lim; i++){
16        if(isprime[i]){
17            primes.push_back(i);
18            for(int j = i*2; j < lim; j+=i){
19                isprime[j] = false;
20            }
21        }
22    }
23
24    //crivo da soma dos divisores no intervalo
25    vector<int> sumdivisor;
26    for(int i = 1; i < lim; i++){
27        for(int j = i; j < lim; j+= i){
28            sumdivisor[j] += i;
29        }
30    }
31
32    //crivo da quantidade de divisores dos nÃºmeros no intervalo
33    vector<int> numdivisors;
34    for(int i = 1; i < lim; i++){
35        for(int j = i; j < lim; j+= i){
36            numdivisors[j]++;
37        }
38    }
39
40    return 0;
41 }

```

6.6 Fft

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long
4 #define endl '\n'
5
6 using cd = complex<double>; // ou long double
7 const double pi = acos(-1);
8
9 void fft(vector<cd> &c, bool invert=false){
10     int n = c.size();
11
12     for(int i = 1, j = 0; i < n; i++){
13         ll bit = n >> 1;
14         for(; j<bit; bit >=>1){
15             j^=bit;
16         }
17         j^=bit;
18
19         if(i < j) swap(c[i], c[j]);
20     }
21
22     for(int len = 2; len<=n; len <=>1){
23         double ang = 2*pi/len * (invert ? -1 : 1);
24         cd wlen(cos(ang), sin(ang));
25
26         for(int i = 0; i < n; i+= len){
27             cd w(1);
28
29             for(int j = 0; j < len/2; j++){
30                 cd u = c[i+j];
31                 cd v = c[i+j+len/2]*w;

```

```

32         c[i+j] = u+v;
33         c[i+j+len/2] = u-v;
34
35         w*=wlen;
36     }
37 }
38
39 }
40
41 if(invert){
42     for(auto &x : c) x /= n;
43 }
44 }
45
46 vector<ll> mul(vector<ll> &a, vector<ll> &b){
47     vector<cd> fa(a.begin(), a.end()), fb(b.begin(),
48         b.end());
49
50     ll n = 1;
51     while(n < (a.size()+b.size())) n<=<=1ll;
52
53     fa.resize(n);fb.resize(n);
54
55     fft(fa, false); fft(fb, false);
56
57     for(int i = 0; i < n; i++) fa[i] *= fb[i];
58
59     fft(fa, true);
60
61     vector<ll> ans(n);
62     for(int i = 0; i < n; i++) ans[i] = round(fa[i].
63         real());
64
65     return ans;
66 }

```

6.7 Fatoracao Prima E Spf

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define ll long long int
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     ll maxn;
11     //SHORTEST PRIME FACTOR
12     vector<ll> spf(maxn+1, 0);
13     ll curr;
14     for(ll i = 2; i <= maxn; i++){
15         if(spf[i] == 0){
16             spf[i] = i;
17             curr = i*i;
18             while(curr <= maxn){
19                 if(spf[curr] == 0){
20                     spf[curr] = i;
21                 }
22                 curr += i;
23             }
24         }
25     }
26
27     //FATORACAO MELHORADA COM SPF -> nlogn
28     ll v;
29     vector<map<ll, int>> fat(maxn+1);
30     for(int i = 2; i <= maxn; i++){
31         v = i;
32         while(v > 1){
33             porfavor[i][spf[v]]++;
34             v = v/spf[v];
35         }

```

```

36     }
37
38     //FATORACAO PRIMA PADRAO -> n * sqrt(n)
39     int n;
40     vector<int> primos;
41     map<int, int> freq;
42     for(int i = 2; i*i <= n; i++){
43         int cnt = 0;
44         while(n % i == 0){
45             n /= i;
46             cnt++;
47         }
48         if(cnt > 0){
49             freq[i] += cnt;
50             primos.push_back(i);
51         }
52     }
53     if(n > 1){
54         primos.push_back(n);
55         freq[n]++;
56     }
57
58     return 0;
59 }

```

6.8 Miller Rabin

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define ll long long
4  #define ull unsigned long long
5  #define sll __uint128_t
6  using namespace std;
7
8  ull fexp(ull a, ull b, ull mod){
9      ull ans = 1;
10     while(b != 0){
11         if(b & 1){
12             ans = ((sll)ans * a) % mod;
13         }
14         a = ((sll)a * a) % mod;
15         b >>= 1;
16     }
17     return ans;
18 }
19
20 bool check_composite(ull n, ull a, ull d, int s) {
21     ull x = fexp(a, d, n);
22     if (x == 1 || x == n - 1)
23         return false;
24     for (int r = 1; r < s; r++) {
25         x = (sll)x * x % n;
26         if (x == n - 1)
27             return false;
28     }
29     return true;
30 };
31
32 //identifica primo ate 1e16
33 bool MillerRabin(ull n) {
34     if (n < 2)
35         return false;
36
37     int r = 0;
38     ull d = n - 1;
39     while ((d & 1) == 0) {
40         d >>= 1;
41         r++;
42     }
43
44     for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
45         31, 37}) {

```

```

46         return true;
47         if (check_composite(n, a, d, r))
48             return false;
49     }
50     return true;
51 }

```

6.9 Lagrange

```

1  const ll maxn = 1e6+5;
2  vector<ll> fat(maxn), invFat(maxn), pref(maxn), suff(
    maxn);
3  const ll MOD = 1e9 + 7;
4
5  ll fexp(ll a, ll b);
6
7  void init_lagrange(int n) {
8      fat[0] = 1;
9      for (int i = 1; i <= n; i++) fat[i] = fat[i - 1]
        * i % MOD;
10     invFat[n] = fexp(fat[n], MOD - 2);
11     for (int i = n - 1; i >= 0; i--) invFat[i] =
        invFat[i + 1] * (i + 1) % MOD;
12 }
13
14 ll interpolate(vector<ll>& y, int T) {
15     int k = y.size() - 1;
16     if (T <= k) return y[T];
17     T %= MOD;
18     pref[0] = 1;
19     for (int i = 0; i <= k; i++) pref[i + 1] = pref[i]
        * (T - i + MOD) % MOD;
20     suff[k + 1] = 1;
21     for (int i = k; i >= 0; i--) suff[i] = suff[i +
        1] * (T - i + MOD) % MOD;
22     ll ans = 0;
23     for (int i = 0; i <= k; i++) {
24         ll num = pref[i] * suff[i + 1] % MOD;
25         ll den = invFat[i] * invFat[k - i] % MOD;
26         if ((k - i) & 1) den = (MOD - den);
27         ans = (ans + y[i] * num % MOD * den) % MOD;
28     }
29     return ans;
30 }

```

6.10 Xor Basis

```

1  #include <bits/stdc++.h>
2  #define ll long long
3  #define endl '\n'
4  using namespace std;
5
6  constexpr ll maxn = 500;
7
8  vector<ll> rep(64, -1);
9  vector<bitset<maxn>> indx(64, -1);
10 //guardar combinacao de indices que gera rep
11 //reconstrucao do valor = xor dos indx que passa no
    basis
12 int pivos = 0;
13 int bad = 0;
14 void basis(ll x){
15     for(int i = 63; i >= 0; i--){
16         if(x&(1LL<<i)){
17             if(rep[i] == -1){
18                 rep[i] = x;
19                 pivos++;
20                 //linearmente independente
21                 return;
22             }
23             x ^= rep[i];
24         }
25     }
26 }

```

```

25     }
26     bad++;
27     //linearmente dependente (redundante)
28     //duplica criacoes dos subsets
29 }
30
31 //coloca na forma escada
32 //necessario para garantir a criacao dos menores
    valores
33 //baseados na ordem dos subsets
34 void escada(){
35     for(int i = 0; i < 64; i++){
36         if(rep[i] != -1){
37             for(int j = 0; j < i; j++){
38                 if(rep[j] != -1 && (rep[i]&(1LL<<j)))
39                     rep[i] ^= rep[j];
40             }
41         }
42     }
43 }
44 }

```

6.11 Ntt E Fexp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define ll long long
4  #define endl '\n'
5
6  const ll mod = 998244353;
7
8  ll fexp(ll a, ll b){
9      ll ans = 1;
10     while(b != 0){
11         if(b & 1){
12             ans = (ans * a) % mod;
13         }
14         a = (a * a) % mod;
15         b >>= 1;
16     }
17     return ans;
18 }
19
20 //pegar g do ntt para um modulo primo p
21 ll get_primitive_root(ll p) {
22     vector<ll> fatores;
23     ll phi = p - 1;
24     ll n = phi;
25
26     for (ll i = 2; i * i <= n; ++i) {
27         if (n % i == 0) {
28             fatores.push_back(i);
29             while (n % i == 0) {
30                 n /= i;
31             }
32         }
33     }
34     if (n > 1) {
35         fatores.push_back(n);
36     }
37
38     for (ll res = 2; res <= p; ++res) {
39         bool ok = true;
40
41         for (ll fator : fatores) {
42             //g^((p-1)/q) tem que ser diferente de 1
43             para todo q dos fatores
44             if (fexp(res, phi / fator) == 1) {
45                 ok = false;
46                 break;
47             }
48         }
49     }
50 }

```

```

48         if (ok) {
49             return res;
50         }
51     }
52     return -1;
53 }
54
55 const ll G=3;
56
57 //g->primitive root (rodar antes)
58 void ntt(vector<ll> &a, bool invert) {
59     int n = a.size();
60
61     for (int i = 1, j = 0; i < n; i++) {
62         int bit = n >> 1;
63         for (; j & bit; bit >>= 1) j ^= bit;
64         j ^= bit;
65         if (i < j) swap(a[i], a[j]);
66     }
67
68     for (int len = 2; len <= n; len <= 1) {
69         ll wlen = fexp(G, (mod - 1) / len);
70
71         if (invert) wlen = fexp(wlen, mod-2);
72
73         for (int i = 0; i < n; i += len) {
74             ll w = 1;
75             for (int j = 0; j < len / 2; j++) {
76                 ll u = a[i + j];
77                 ll v = (a[i + j + len / 2] * w) % mod
78             ;
79                 a[i + j] = (u + v < mod ? u + v : u +
80 v - mod);
81                 a[i + j + len / 2] = (u - v >= 0 ? u
82 - v : u - v + mod);
83                 w = (w * wlen) % mod;
84             }
85         }
86     }
87
88     if (invert) {
89         ll n_inv = fexp(n, mod-2);
90         for (ll &x : a) x = (x * n_inv) % mod;
91     }
92 }
93
94 vector<ll> mul(vector<ll> const& a, vector<ll> const&
95 b) {
96     vector<ll> fa(a.begin(), a.end()), fb(b.begin(),
97 b.end());
98
99     int n = 1;
100     while (n < a.size() + b.size()) n <= 1;
101
102     fa.resize(n);
103     fb.resize(n);
104
105     ntt(fa, false);
106     ntt(fb, false);
107
108     for (int i = 0; i < n; i++)
109         fa[i] = (fa[i] * fb[i]) % mod;
110
111     ntt(fa, true);
112
113     while (fa.size() > 1 && fa.back() == 0) fa.
114 pop_back();
115
116     return fa;
117 }

```

```

115
116 vector<ll> polyfexp(vector<ll> &base, ll exp) {
117     vector<ll> res = {1};
118
119     while (exp != 0) {
120         if (exp & 1) {
121             res = mul(res, base);
122         }
123         base = mul(base, base);
124         exp >>= 1;
125     }
126
127     return res;
128 }

```

6.12 Eliminacao Gaussiana

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long
4 using namespace std;
5
6 const ll mod;
7
8 ll fexp(ll a, ll b);
9
10 int gauss(int n, int m, vector<vector<ll>> &mat){
11     int l = 0; //rank
12     ll pivo;
13     //col
14     for(int i = 0; i < m; i++){
15         pivo = -1;
16         for(int j = 1; j < n; j++){
17             if(mat[j][i] != 0){
18                 pivo = j;
19                 break;
20             }
21         }
22         if(pivo==-1) continue;
23         swap(mat[pivo], mat[l]);
24         //IGNORAR NO CASO DE MOD 2 vvvv
25         ll denom = fexp(mat[l][i], mod-2);
26         for(int k = 0; k < m; k++){
27             if(mat[l][k] != 0){
28                 mat[l][k] = (mat[l][k] * denom) % mod
29             ;
30         }
31         //IGNORAR NO CASO DE MOD 2 ^^^^
32
33         for(int j = 0; j < n; j++){
34             if(j!=l){
35                 ll mul = mat[j][i];
36                 for(int k = 0; k < m; k++){
37                     mat[j][k] = (((mat[j][k] - ((mat[
38 l][k] * mul) % mod)) % mod) + mod) % mod;
39                 }
40             }
41         }
42         //CASO SEJA MOD 2
43         //usar bitsets
44         // for(int j = 0; j < n; j++){
45         //     if(j!=l){
46         //         ll mul = mat[j][i];
47         //         if(mul){
48         //             mat[j] ^= mat[l];
49         //         }
50         //     }
51         // }
52         l++;
53     }
54     //retorna rank da matriz
55     return l;

```

55 }

6.13 Matrices

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 vector<vector<long long int>> mult(vector<vector<long
    long int>> &a, vector<vector<long long int>> &b,
    long long int MOD){
6     vector<vector<long long int>> res(a.size(),
    vector<long long int>(b[0].size()));
7
8     for(int i = 0; i < a.size(); i++){
9         for(int j = 0; j < b[0].size(); j++){
10             res[i][j] = 0;
11             for(int k = 0; k < a[0].size(); k++){
12                 res[i][j] += (a[i][k] * b[k][j]) %
    MOD;
13             }
14             res[i][j] = res[i][j] % MOD;
15         }
16     }
17     return res;
18 }
19 }
20
21 vector<vector<long long int>> fexp(vector<vector<long
    long int>> &a, long long int e, long long int
    MOD){
22     vector<vector<long long int>> ans(4, vector<long
    long int>(4, 0));
23     ans[0][0] = 1;
24     ans[1][1] = 1;
25     ans[2][2] = 1;
26     ans[3][3] = 1;
27
28     while(e){
29         if(e & 1){
30             ans = mult(a, ans, MOD);
31         }
32         a = mult(a, a, MOD);
33         e >>= 1;
34     }
35
36     return ans;
37 }
38
39 int main() {
40     ios_base::sync_with_stdio(false);
41     cin.tie(NULL);
42
43     //recorrecia em matrizes
44     //matrix T * matrix (f0) = matrix (f2)
45     //                (f1)          (f1)
46
47     return 0;
48 }

```

7 Geometria

7.1 Line Container

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define ll long long
4 using namespace std;
5
6 const ll inf = 1e18;
7

```

```

8 struct line{
9     ll a, b;
10     mutable ll end_segment;
11
12     bool operator<(const line &l) const{
13         return tie(a, b) > tie(l.a, l.b);
14     }
15
16     bool operator<(const ll x) const{
17         return end_segment < x;
18     }
19 };
20
21 struct LineContainer : multiset<line, less<>>{
22
23     ll intersect(iterator i, iterator j){
24         ll x = j->b - i->b;
25         ll y = i->a - j->a;
26         //usar somente x/y para doubles, ajustar inf
27         return x/y - (((x~y)<0) and ((x%y) != 0));
28     }
29
30     void update(iterator i){
31         if(next(i) == end()){
32             i->end_segment = inf;
33         } else{
34             i->end_segment = intersect(i, next(i));
35         }
36     }
37
38     void add(ll a, ll b){
39         iterator l = insert({a, b, 0});
40
41         if(l != begin() and (prev(l)->a == l->a)){
42             erase(prev(l));
43         }
44         if(next(l) != end() and (next(l)->a == l->a))
45     {
46         erase(l);
47         return;
48     }
49
50     if(l!=begin() and (next(l) != end())){
51         iterator p = prev(l);
52         iterator n = next(l);
53         if(intersect(p, l) >= intersect(l, n)){
54             erase(l);
55             return;
56         }
57     }
58
59     while(l!=begin()){
60         iterator p = prev(l);
61         if((p!=begin() and (intersect(prev(p), p
62 ) >= intersect(p, l))){
63             erase(p);
64             continue;
65         }
66         break;
67     }
68
69     while(next(l) != end()){
70         iterator n = next(l);
71         if(next(n) != end() and (intersect(l, n
72 ) >= intersect(n, next(n)))){
73             erase(n);
74             continue;
75         }
76         break;
77     }
78
79     if(l!=begin()){
80         update(prev(l));
81     }
82 }

```

```

78     }
79     update(l);
80 }
81
82 ll min(ll x){
83     auto l = lower_bound(x);
84     return l->a * x + l->b;
85 }
86 };

```

7.2 Ponto

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5  //trocar os long longs por outro tipo desejado, ou:
6  //template<typename T>
7  struct Point{
8      long long int x, y;
9      Point(long long int x, long long int y){
10         this->x = x;
11         this->y = y;
12     }
13     Point operator+(Point o){ return Point(x + o.x, y
14         + o.y); }
15     Point operator-(Point o){ return Point(x - o.x, y
16         - o.y); }
17     Point operator*(long long int k){ return Point(k*
18         x, k*y); }
19     double len(){ return hypot(x, y); }
20     long long int cross(Point o){ return ((x * o.y) -
21         (y*o.x)); }
22     bool operator<(Point o){ return(tie(x, y) < tie(o
23         .x, o.y)); }
24     bool operator==(Point o){ return (tie(x, y) ==
25         tie(o.x, o.y)); }
26 };
27
28 int main() {
29     ios_base::sync_with_stdio(false);
30     cin.tie(NULL);
31
32     int t;
33     long long int x1, x2, x3, y1, y2, y3;
34     for(int k = 0; k < t; k++){
35         Point p1 = Point(x2 - x1, y2 - y1);
36         Point p2 = Point(x3 - x1, y3 - y1);
37         //produto vetorial
38         long long int check = p2.cross(p1);
39         if(check == 0){
40             cout << "TOUCH" << endl;
41         } else if(check > 0){
42             cout << "RIGHT" << endl;
43         } else{
44             cout << "LEFT" << endl;
45         }
46     }
47
48     //numerador = abs(((y2 - y) * cx) - ((x2 - x) *
49     cy) + (x2 * y) - (y2 * x));
50     //denominador = sqrt(((y2 - y) * (y2 - y)) + ((x2
51     - x) * (x2 - x)));
52     //dist = numerador/denominador;
53     //distancia entre reta formada pelos pontos (x, y
54     ) e (x2, y2) ate o ponto (cx, cy)
55
56     return 0;
57 }

```

7.3 Convex Hull

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5  struct Point{
6      //alterar tipos se preciso
7      long long int x, y;
8      Point(long long int x, long long int y){
9         this->x = x;
10         this->y = y;
11     }
12     Point operator+(Point o){ return Point(x + o.x, y
13         + o.y); }
14     Point operator-(Point o){ return Point(x - o.x, y
15         - o.y); }
16     Point operator*(long long int k){ return Point(k*
17         x, k*y); }
18     double len(){ return hypot(x, y); }
19     long long int cross(Point o){ return ((x * o.y) -
20         (y*o.x)); }
21     bool operator<(Point o){ return(tie(x, y) < tie(o
22         .x, o.y)); }
23     bool operator==(Point o){ return (tie(x, y) ==
24         tie(o.x, o.y)); }
25 };
26
27 int orientation(Point a, Point b, Point c){
28     Point ab = b - a;
29     Point bc = c - b;
30     long long int v = ab.cross(bc);
31
32     if(v < 0){
33         return -1; //horario
34     }
35     if(v > 0){
36         return 1; //antihorario
37     }
38     return 0;
39 }
40
41 bool cw(Point a, Point b, Point c, bool
42     include_collinear){
43     int o = orientation(a, b, c);
44     return ((o < 0) || (include_collinear && (o == 0)
45     ));
46 }
47
48 bool ccw(Point a, Point b, Point c, bool
49     include_collinear){
50     int o = orientation(a, b, c);
51     return ((o > 0) || (include_collinear && (o == 0)
52     ));
53 }
54
55 void convex_hull(vector<Point> &a, bool
56     include_collinear=false){
57     if(a.size() == 1){
58         return;
59     }
60     sort(a.begin(), a.end());
61     Point p1 = a[0];
62     Point p2 = a.back();
63     vector<Point> up, down;
64     up.push_back(p1);
65     down.push_back(p1);
66
67     for(int i = 1; i < (int)a.size(); i++){
68         if((i == a.size() - 1) || (cw(p1, a[i], p2,
69         include_collinear))){
70             while((up.size() >= 2) && !(cw(up[up.size
71         () - 2], up[up.size() - 1], a[i],
72         include_collinear))){
73                 up.pop_back();
74             }
75         }
76     }
77 }

```

```

60         }
61         up.push_back(a[i]);
62     }
63     if((i == a.size() - 1) || (ccw(p1, a[i], p2,
include_collinear))) {
64         while((down.size() >= 2) && !(ccw(down[
down.size() - 2], down[down.size() - 1], a[i],
include_collinear))) {
65             down.pop_back();
66         }
67         down.push_back(a[i]);
68     }
69 }
70 if(include_collinear && (up.size() == a.size())) {
71     reverse(a.begin(), a.end());
72     return;
73 }
74 a.clear();
75 for(int i = 0; i < (int)up.size(); i++) {
76     a.push_back(up[i]);
77 }
78 for(int i = down.size() - 2; i > 0; i--) {
79     a.push_back(down[i]);
80 }
81 }
82
83 bool insidetriangle(Point a, Point b, Point c, Point
point) {
84     long long int s1 = abs((b-a).cross(c-b));
85     long long int area1 = abs((point - a).cross(point
- b));
86     long long int area2 = abs((point - b).cross(point
- c));
87     long long int area3 = abs((point - c).cross(point
- a));
88     long long int s2 = area1 + area2 + area3;
89     return s1 == s2;
90 }
91
92 bool isinside(vector<Point> &hull, Point p){
93     int n = hull.size();
94     if(n == 1){
95         return (hull.front() == p);
96     }
97     int l = 1;
98     int r = n - 1;
99     int mid;
100     while(abs(r - l) > 1){
101         mid = (r+l)/2;
102         Point tomid = hull[mid] - hull[0];
103         Point topoint = p - hull[0];
104         if(topoint.cross(tomid) < 0){
105             //a esquerda
106             r = mid;
107         } else{
108             l = mid;
109         }
110     }
111     //Point vec = hull[r] - hull[l];
112     //Point tovec = p - hull[l];
113     //return (tovec.cross(vec) > 0);
114     return insidetriangle(hull[0], hull[l], hull[r],
p);
115 }
116
117 int main() {
118     ios_base::sync_with_stdio(false);
119     cin.tie(NULL);
120
121     return 0;
122 }

```